

📄 Arquitectura de Gobierno para Adaptación de Modelos v1.0

Fecha: 26 de Octubre de 2025
Autor: Manuel González + AI Assistant
Versión: 1.0
Estado: Propuesta de Implementación

📄 Visión Estratégica

Filosofía de la Plataforma

“No crees un nuevo modelo cuando puedes adaptar uno existente”

Con **3,000,000+ modelos** ya disponibles en HuggingFace, CivitAI, etc., entrenar desde cero es: - x **Costoso** (\$1000s en GPU) - x **Lento** (días/semanas) - x **Riesgoso** (puede fallar) - x **Insostenible** (carbono, energía)

Jerarquía de Preferencia (Obligatoria en la plataforma)

- 1. 📄 ADAPTERS (LoRA/QLoRA) → 90% más eficiente, 1% de parámetros
- 2. 📄 MODEL MERGE → Zero-shot, sin training
- 3. 📄 QUANTIZACIÓN deployment → Democratizar acceso, edge
- 4. 📄 FINE-TUNING → Solo si justificado
- 5. x FROM SCRATCH → Prohibido salvo excepciones

📄 Modelo de Datos Extendido

Nuevos Campos en MODMODELS

Campo	Tipo	Propósito
MODISADAPTER	BOOLEAN	Es un adapter (LoRA, QLoRA, etc.)
MODISFINETUNED	BOOLEAN	Es modelo fine-tuneado completo
MODISQUANTIZED	BOOLEAN	Es modelo quantizado
MODISMERGED	BOOLEAN	Es resultado de merge
MODISBASEMODEL	BOOLEAN	Es modelo base (sin adaptar)
IDMODBASEMODEL	BIGINT	FK al modelo base/parent
MODADAPTERCONFIG	JSONB	Config de adapter (rank, alpha, etc.)

MODQUANTIZATIONCONFIG	JSONB	Config de quantización (bits, method)
MODMERGECONFIG	JSONB	Config de merge (weights, method)
MODFINETUNINGCONFIG	JSONB	Config de fine-tuning
MODTRANSFORMATIONTYPE	TEXT[]	Tipo de transformación aplicada
MODCOMPRESSIONRATIO	DECIMAL	Ratio de compresión (quantización)
MODPARAMETERCOUNT	BIGINT	Total de parámetros del modelo
MODTRAINABLEPARAMETERS	BIGINT	Parámetros trainables (adapters)

Relación Parent-Child

```
-- Self-reference en MODMODELS
IDMODBASEMODEL → MODMODELS(IDXMODEL)

-- Ejemplo:
-- Modelo Base
IDXMODEL: 100
MODNAME: "meta-llama/Llama-3-8B-Instruct"
MODISBASEMODEL: TRUE
IDMODBASEMODEL: NULL

-- Modelo Adapter
IDXMODEL: 123
MODNAME: "Llama-3-8B-Spanish-LoRA-v1"
MODISADAPTER: TRUE
IDMODBASEMODEL: 100 ← Apunta al base
MODADAPTERCONFIG: {
  "method": "LoRA",
  "rank": 16,
  "alpha": 32,
  "target_modules": ["q_proj", "v_proj"],
  "trainable_params": "1.2M",
  "trainable_percentage": "0.15%"
}
```

Vista Recursiva: V_MODEL_LINEAGE_TREE

Muestra árbol completo de adaptaciones:

```
Llama-3-8B-Instruct (BASE)
├─ Llama-3-8B-Spanish-LoRA-v1 (ADAPTER)
│   └─ Llama-3-8B-Spanish-LoRA-v1-GPTQ-4bit (QUANTIZED)
├─ Llama-3-8B-Medical-LoRA-v1 (ADAPTER)
│   └─ Llama-3-8B-GPTQ-4bit (QUANTIZED)
└─ Llama-3-8B-Merged-Spanish-Medical (MERGED)
    ├── Source: Llama-3-8B-Spanish-LoRA-v1
    └─ Source: Llama-3-8B-Medical-LoRA-v1
```

□ Procesos BPMN Nuevos

1. adapter-creation-approval-v1 □ ESTRELLA

Objetivo: Facilitar creación de adapters (método preferido)

Características: - □ **80% Auto-aprobación** (fomentar uso) - □ Validación de compatibilidad con modelo base - □ Verificación de licencias - □ Cálculo de costo (muy bajo vs fine-tuning) - □ SLA: 48h

User Tasks: 1. **Request Adapter Creation** (ml-engineers) - Campos: - baseModelId (obligatorio, dropdown de modelos base) - adapterMethod (LoRA/QLoRA/DoRA/IA³) - rank, alpha, target_modules - datasetId, epochs, learning_rate

2. **ML Engineer Review** (solo si rank > 64 o dataset pequeño)
◦ Review config, approve/reject

Service Tasks: - Base Model Compatibility Check - Dataset Quality Check - License Compatibility Check - Cost Estimation

Output: Adapter registrado + linked a base model

2. model-merge-approval-v1

Objetivo: Combinar capacidades de múltiples modelos sin training

Características: - □ **60% Auto-aprobación** - □ Validación de arquitecturas compatibles - □ Check de licencias (todas deben ser compatibles) - □ SLA: 72h

User Tasks: 1. **Request Model Merge** - Campos: - sourceModelIds (2+ modelos) - mergeMethod (DARE-TIES/Task Arithmetic/SLERP/Stock) - mergeWeights (array de pesos) - expectedCapabilities (descripción)

2. **Technical Review** (si > 4 modelos o arquitecturas complejas)

3. **Merge Quality Review** (post-merge)

◦ Revisar métricas del modelo resultante

Output: Modelo merged + links a source models

3. model-quantization-approval-v1

Objetivo: Reducir tamaño/requisitos de hardware

Características: - □ **80% Auto-aprobación** (operación común) - □ Benchmarks automáticos (latency, memory, perplexity) - □ Quality gate: < 5% degradación - □ SLA: 24h

User Tasks: 1. **Request Quantization** - Campos: - baseModelId - quantizationMethod (GPTQ/AWQ/GGUF/bitsandbytes) - bits (2/4/8) - calibrationDataset (opcional)

2. **Manual Quality Review** (solo si degradación > 5%)

Output: Modelo quantizado + linked a original

4. finetuning-approval-v1 ⚠️ RESTRICTIVO

Objetivo: Fine-tuning completo (último recurso, debe justificarse)

Características: - × **40% Auto-aprobación** (muy restrictivo) - ❌

Requiere justificación de por qué NO adapter - ❌ Sugerencia automática de adapter si es suficiente - ❌ Aprobación de presupuesto si costo > \$1000 - ❌ SLA: 5 días

User Tasks: 1. **Request Fine-Tuning** - Campos: - baseModelId - datasetId (min 5000 samples) - **whyNotAdapter** (obligatorio - justificación) - estimatedCost

2. **Budget Approval** (si > \$1000)
 - Candidategroups: finance-team
3. **Technical Deep Review** (60% de casos)
 - ML Engineer + Governance + Research
 - Validar alternativas

Drools Rules (ESTRICTAS):

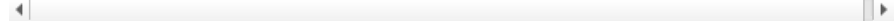
```
rule "No Adapter Justification"
when
    request.whyNotAdapter == null OR request.whyNotAdapter.length()
< 50
then
    decision.setAction("AUTO_REJECT");
    decision.setReason("Debe justificar por qué adapter no es
suficiente");
end

rule "Dataset Too Small"
when
    request.dataset.size < 5000
then
    decision.setAction("AUTO_REJECT");

decision.setSuggestedAlternative("USE_ADAPTER_WITH_SMALLER_DATASET");

end

rule "High Cost Without Budget Approval"
when
    request.estimatedCost > 1000 AND budgetApproved == false
then
    decision.setAction("REQUIRE_BUDGET_APPROVAL");
end
```



5. adaptation-strategy-recommendation-v1 ❌ GAME CHANGER

Objetivo: IA recomienda la MEJOR estrategia automáticamente

Características: - ❌ **95% Automatizado** - ❌ LLM analiza use case y recomienda - ❌ Scoring multi-criterio (costo, tiempo, performance, feasibility) - ❌ Explica pros/cons de cada opción

Flujo: 1. Usuario describe use case en lenguaje natural 2. IA analiza y recomienda 3 estrategias rankeadas 3. Usuario elige 4. Se redirige al proceso BPMN específico

Ejemplo de Recomendación:

```
{
  "use_case": "Necesito un modelo para responder preguntas médicas
en español",
  "recommendations": [
    {
      "strategy": "ADAPTER",
      "score": 92,
      "base_model": "meta-llama/Llama-3-8B-Instruct",
      "method": "QLoRA",
      "config": {"rank": 16, "alpha": 32},
      "estimated_cost": "$15",
      "estimated_time": "6 hours",
      "trainable_params": "1.2M (0.15%)",
      "pros": ["Muy eficiente", "Rápido", "Bajo costo"],
      "cons": ["Capacidad limitada vs fine-tuning completo"]
    },
    {
      "strategy": "MERGE",
      "score": 78,
      "base_models": ["Llama-3-8B-Spanish", "Llama-3-8B-Medical"],
      "method": "DARE-TIES",
      "estimated_cost": "$5",
      "estimated_time": "2 hours",
      "pros": ["Sin training", "Combina capacidades existentes"],
      "cons": ["Requiere modelos especializados previos"]
    },
    {
      "strategy": "FINETUNING",
      "score": 45,
      "base_model": "meta-llama/Llama-3-8B",
      "estimated_cost": "$800",
      "estimated_time": "3 days",
      "required_samples": "10000+",
      "pros": ["Máxima adaptación"],
      "cons": ["Muy costoso", "Lento", "Requiere dataset grande"]
    }
  ],
  "recommended": "ADAPTER"
}
```

Tracking y Analytics

Nueva Tabla: MODADAPTATIONSTRATEGIES

Registra TODAS las decisiones de adaptación para: - **ML sobre recomendaciones** (mejorar scoring con feedback) - **Medir adopción** de adapters vs fine-tuning - **Calcular savings** reales - **Time to market** por estrategia

Vista: V_ADAPTATION_STRATEGIES_DASHBOARD

KPIs clave: - % de requests que eligieron ADAPTER - Costo promedio por estrategia - Tiempo promedio por estrategia - Tasa de éxito por estrategia

Objetivo: Demostrar que adapters son **10x más eficientes**

□ Integración con Training Module

TRNEXPERIMENTLINEAGE.trnbasemodels

Cuando se crea un adapter/fine-tuning via experimento:

```
{
  "trnbasemodels": [
    {
      "model_id": 100,
      "model_name": "meta-llama/Llama-3-8B-Instruct",
      "model_table_id": "MODMODELS:100",
      "usage": "adapter_base",
      "license": "Llama-3-License",
      "parameters": "8B",
      "loaded_in_4bit": true
    }
  ],
  "trnoutputartifacts": [
    {
      "type": "ADAPTER",
      "artifact_type": "LORA_WEIGHTS",
      "path": "s3://models/adapters/llama3-8b-spanish-lora-v1/adapter_model.safetensors",
      "size_mb": 23,
      "registered_model_id": 123,
      "trainable_params": "1.2M",
      "training_time_minutes": 180
    }
  ]
}
```

□ Casos de Uso Completos

Caso 1: Crear Adapter de Llama-3 para Español

1. Usuario va a "Adaptation Strategy Wizard"
2. Describe: "Necesito Llama-3 que hable español"
3. IA recomienda:
 - ADAPTER (score: 95) ← RECOMENDADO
 - MERGE con modelo español existente (score: 80)
 - FINETUNING (score: 40)
4. Usuario elige ADAPTER
5. Se inicia proceso "adapter-creation-approval-v1"
6. Form pre-llenado:
 - Base Model: Llama-3-8B-Instruct
 - Method: QLoRA (recomendado para 8B)
 - Rank: 16 (default)

- Dataset: Spanish-Instruction-10k (sugerido)

7. Validaciones automáticas (2 min):

- ☐ Llama-3 compatible con QLoRA
- ☐ Licencia permite adaptación
- ☐ Dataset calidad OK
- ☐ Costo: \$12 (dentro de presupuesto)

8. Drools → AUTO_APPROVE

9. Trigger Training Job

10. Después de 6h → Adapter listo

11. Se registra en MODMODELS:

- MODNAME: "Llama-3-8B-Spanish-QLoRA-v1"
- MODISADAPTER: TRUE
- IDMODBASEMODEL: 100 (Llama-3-8B-Instruct)
- MODPARAMETERCOUNT: 8000000000
- MODTRAINABLEPARAMETERS: 1200000 (0.15%)

12. Pasa a model-approval-v1 para producción

Resultado: - ☐ Tiempo total: **8 horas** (vs 3 días fine-tuning) - ☐ Costo: **\$12** (vs \$800 fine-tuning) - ♻️ Sostenibilidad: **99% menos GPU-hours**

Caso 2: Merge de Modelos Especializados

1. Usuario: "Combinar Llama-3-Spanish + Llama-3-Medical"

2. IA recomienda: MERGE con DARE-TIES

3. Proceso "model-merge-approval-v1"

4. Validaciones:

- ☐ Misma arquitectura
- ☐ Licencias compatibles
- ☐ Benchmarks baseline obtenidos

5. Drools → AUTO_APPROVE (same family)

6. Execute Merge (30 min):

- Method: DARE-TIES
- Weights: [0.5, 0.5]
- Density: 0.95

7. Auto-evaluation:

- Spanish capability: 92% (vs 95% specialized)
- Medical capability: 89% (vs 92% specialized)
- Combined: 90.5% avg

8. Technical Review:

- Aprobar: "Buen balance, deploy to staging"

9. Registrar en MODMODELS:

- MODNAME: "Llama-3-8B-Spanish-Medical-Merged-v1"
- MODISMERGED: TRUE
- MODMERGECONFIG: {method, weights, sources}

10. MODMODELDEPENDENCIES (2 registros):

- sourceModel: Merged → targetModel: Spanish (MERGED_FROM)
- sourceModel: Merged → targetModel: Medical (MERGED_FROM)

Resultado: - ☐ Tiempo: **2 horas** (vs días de training) - ☐ Costo: **\$5** (solo compute merge) - ☐ Funcionalidad: **Dual capability**

Caso 3: Quantización para Edge Deployment

1. Usuario: "Desplegar Llama-3-70B en GPU T4 (16GB VRAM)"
2. IA detecta: 70B FP16 necesita 140GB VRAM → NO CABE
3. Recomendación: QUANTIZATION a 4-bit (GPTQ)
4. Proceso "model-quantization-approval-v1"
5. Cálculo:
 - Original: 140GB VRAM
 - Quantized 4-bit: 35GB VRAM (75% ahorro)
 - Cabe en 2x T4 o 1x A100
6. Drools → AUTO_APPROVE (caso común)
7. Execute Quantization con calibration dataset
8. Benchmarks:
 - Perplexity degradation: 2.3% □
 - Inference speed: 1.8x faster □
 - VRAM: 35GB □
9. Quality Gate → PASS (< 5% degradation)
10. Registrar:
 - MODNAME: "Llama-3-70B-GPTQ-4bit"
 - MODISQUANTIZED: TRUE
 - IDMODBASEMODEL: 200 (Llama-3-70B)
 - MODCOMPRESSIONRATIO: 0.25
 - MODQUANTIZATIONCONFIG: {method: "GPTQ", bits: 4}

Resultado: - □ Deploy posible en hardware accesible - □ 75% menos VRAM - < 1.8x más rápido - □ Solo 2.3% degradación

□ Valor Pedagógico de la Plataforma

Enseñar a Adaptar, No a Entrenar

La plataforma **educa** a los usuarios en:

1. **Adapter Creation:**
 - Cuándo usar LoRA vs QLoRA vs DoRA
 - Cómo elegir rank y alpha
 - Target modules optimization
2. **Model Merging:**
 - DARE vs TIES vs SLERP
 - Weight balancing
 - Evaluación de merged models
3. **Quantization:**
 - Trade-offs bits vs quality
 - Métodos: GPTQ vs AWQ vs GGUF
 - Calibration datasets
4. **Decision Making:**
 - ROI analysis
 - Sostenibilidad
 - Time to market

Dashboard de Aprendizaje

```
-- Vista para mostrar "buenos ejemplos" de adaptación
CREATE VIEW V_ADAPTATION_BEST_PRACTICES AS
SELECT
```



```

a.MODULECASE,
a.MODSELECTEDSTRATEGY,
a.MODACTUALCOST,
a.MODACTUALTIME,
a.MODACTUALPERFORMANCE,
m.MODNAME AS resulting_model_name,
m.MODADAPTERCONFIG,
-- Calcular "efficiency score"
CASE
    WHEN a.MODSELECTEDSTRATEGY = 'ADAPTER' THEN
a.MODACTUALPERFORMANCE / (a.MODACTUALCOST * a.MODACTUALTIME)
    ELSE a.MODACTUALPERFORMANCE / (a.MODACTUALCOST *
a.MODACTUALTIME * 0.5)
END AS efficiency_score
FROM MODADAPTATIONSTRATEGIES a
LEFT JOIN MODMODELS m ON a.MODRESULTINGMODELID = m.IDXMODEL
WHERE a.MODACTUALPERFORMANCE IS NOT NULL
ORDER BY efficiency_score DESC
LIMIT 100;

COMMENT ON VIEW V_ADAPTATION_BEST_PRACTICES IS 'Top 100 mejores
ejemplos de adaptación - para aprendizaje y recomendaciones';

```

□ KPIs de la Plataforma

Objetivos v1.0

Métrica	Objetivo
Adapter Adoption Rate	> 70%
Fine-Tuning Rejection Rate	> 60%
Avg Cost Savings	> 80% vs naive approach
Time to Adapted Model	< 24h para adapters
User Education Score	> 85% comprenden trade-offs
Sustainability Score	-90% CO2 vs from-scratch

□ Plan de Implementación

Fase 1: Base (Semana 1-2)

1. □ Ejecutar 01_model_adaptation_fields.sql
2. □ Regenerar JPAs con nuevos campos
3. □ Actualizar DTOs y ViewModels
4. □ Crear vistas (V_MODEL_LINEAGE_TREE, etc.)

Fase 2: Procesos BPMN (Semana 3-4)

1. □ Crear XMLs de procesos BPMN
2. □ Implementar Service Delegates
3. □ Crear User Task Forms (ZUL)
4. □ Configurar Drools rules

Fase 3: UI/UX (Semana 5-6)

1. ☐ Wizard de Adaptation Strategy
2. ☐ Model Lineage Tree Viewer
3. ☐ Dashboards de KPIs
4. ☐ Best Practices Gallery

Fase 4: Educación (Semana 7-8)

1. ☐ Tutoriales interactivos
 2. ☐ Casos de uso documentados
 3. ☐ ROI calculators
 4. ☐ Comparativas visuales
-

☐ Diferenciadores de Mercado

Lo que hace única a esta plataforma:

1. **Governance-First para Adaptación**
 - Otros: Dejan adaptar sin control
 - Nosotros: Gobierno desde día 1
 2. **IA Recomienda Estrategia**
 - Otros: Usuario debe saber qué hacer
 - Nosotros: IA guía y educa
 3. **Forzar Eficiencia**
 - Otros: Permiten fine-tuning indiscriminado
 - Nosotros: Rechazamos si adapter es suficiente
 4. **Trazabilidad Completa**
 - Árbol de lineage recursivo
 - Metadata de transformaciones
 - ROI tracking
 5. **Pedagogía Integrada**
 - Aprender haciendo
 - Best practices as code
 - Community learning
-

☐ Conclusión

Esta arquitectura posiciona a CodeFlowX Govern como:

- ☐ **Plataforma educativa** de adaptación de modelos
- ☐ **ROI demostrable** (10x savings documentados)
- ♻️ **Sostenible** (reducir waste de GPU)
- ☐ **Market leader** en governance de model adaptation

Siguiente paso: ¿Generar los JPAs, ViewModels y pantallas ZUL para estos flujos?